

# Engineering Challenge: Mission-Critical Incident Management System (IMS)

## 1. Overview

The goal of this assignment is to build a resilient **Incident Management System (IMS)** designed to monitor a complex distributed stack (**APIs, MCP Hosts, Distributed Caches, Async Queues, RDBMS, and NoSQL stores**) and manage failure mediation workflow.

In a production environment, "signals" (errors/latency spikes) arrive in high volumes. Your system must intelligently ingest these signals, process and store them, alert the right responders, and provide a workflow-driven UI to track the incident to a "Closed" state with a mandatory **Root Cause Analysis (RCA)**.

## 2. Technical Architecture

### A. Ingestion & In-Memory Processing (The Producer)

- **Signal Ingestion:** Support high-throughput ingestion of signals (hint: choose the right protocol and formats).
- **Memory Management:** The system must handle bursts of up to 10,000 signals/sec. (hint: your system cannot crash if persistence layer is slow)
- **Debouncing Logic:** If 100 signals arrive for the same "Component ID" (e.g., `CACHE_CLUSTER_01`) within 10 seconds, only **one** Work Item should be created, while all 100 signals are linked to it in the NoSQL store.

### B. Distribution & Persistence (The Storage)

- **Sink (The Data Lake):** Store the high-volume, raw error payloads. This acts as the "audit log" for every signal. (Hint: think how this can be queried)
- **Sink (The Source of Truth):** Store the structured **Work Items** and **RCA records**. Transitions here must be transactional.
- **Cache (The Hot-Path):** Maintain a "Real-time Dashboard State" to avoid querying the Source of truth for every UI refresh.
- **Sink (Aggregations):** Support timeseries aggregations.

### C. The Workflow Engine (Strategy & State Patterns)

Implement the incident lifecycle using robust design patterns:

- **Alerting Strategy:** Different component failures require different alert types (e.g., P0 for RDBMS failure, P2 for Cache failure). Use the **right Design Pattern** to swap alerting logic.
- **Work Item State:** Manage transitions (OPEN → INVESTIGATING → RESOLVED → CLOSED) using the **the right Design Pattern**.

### 3. Functional Requirements

#### The Backend Engine

- 1. **Async Processing:** The system must operate on Async processing.
- 2. **Mandatory RCA:** The system must reject any attempt to move a Work Item to CLOSED if the RCA object is missing or incomplete.
- 3. **MTTR Calculation:** The system must automatically calculate the **Mean Time To Repair** based on the start\_time (first signal) and end\_time (RCA submission).

#### The Incident Dashboard (UI)

You are required to build a simple, responsive **Frontend (React, Vue, or HTMX)** that allows:

- **Live Feed:** View active incidents sorted by severity.
- **Incident Detail:** Click an incident to see the raw signals (from NoSQL) and the current status.
- **RCA Form:** A dedicated interface to fill out:
  - Incident Start/End (Date-time pickers).
  - Root Cause Category (Dropdown).
  - Fix Applied & Prevention Steps (Text areas).

### 4. Technical Constraints & Resilience

- **Concurrency:** Use modern concurrency primitives.
- **Rate Limiting:** Implement a rate-limiter on the Ingestion API to prevent cascading failures.
- **Observability:** Expose a /health endpoint and print throughput metrics (Signals/sec) to the console every 5 seconds.

### 5. Evaluation Rubric

| Category              | Weight | Criteria                                                                          |
|-----------------------|--------|-----------------------------------------------------------------------------------|
| Concurrency & Scaling | 10%    | Proper handling of high-volume signals. No race conditions during status updates. |
| Data Handling         | 20%    | Correct separation of data for various purpose.                                   |
| LLD                   | 20%    | Use of best practise code constructs in the language of choice.                   |

|                      |     |                                                                                    |
|----------------------|-----|------------------------------------------------------------------------------------|
| UI/UX & Integration  | 20% | Functional dashboard that correctly interacts with the backend APIs.               |
| Resilience & Testing | 10% | Evidence of retry logic for DB writes and unit tests for the RCA validation logic. |
| Documentation        | 10% | Comprehensive readme and markdown files.                                           |
| Tech Stack choices   | 10% | System design of tech stack                                                        |

## 6. Submission Guidelines

1. **Codebase:** A single repository containing /backend and /frontend.
2. **README.md:** Must include an **Architecture Diagram**, setup instructions (Docker Compose), and a section on how you handled **Backpressure**.
3. **Sample Data:** Provide a script or JSON file to mock a failure event across the stack (e.g., simulating an RDBMS outage followed by an MCP failure).
4. **Prompts/Spec/Plans:** All markdowns and prompts used to create this repository should be checked in.
5. **Bonus points for any creative additions done.**

**Submission Deadline:** 1 week from receipt.